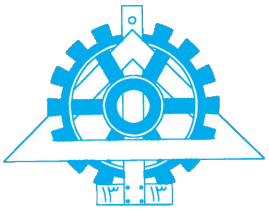
University of Tehran
College of Engineering
School of ECE



Machine Learning

Assignment #2

Student Name
Amirali Dehghani

Student ID
810102443

November 24, 2025

Contents

List of Tables

Abstract

Optimization lies at the heart of machine learning, serving as the mathematical engine for training models and making decisions. This assignment provides a comprehensive exploration of fundamental and advanced optimization techniques, ranging from classical gradient-based methods to heuristic and multi-objective approaches.

The study begins with an analysis of unconstrained optimization, specifically **Line Search** methods, establishing the conditions for descent directions and optimal step lengths. We then transition to constrained optimization through the rigorous derivation of the **Support Vector Machine (SVM)**, exploring the Primal and Dual forms, Lagrangian multipliers, and the geometric interpretation of KKT conditions and Support Vectors.

Furthermore, the assignment delves into advanced scenarios such as **Bilevel Optimization**, modeled as a Stackelberg game between a leader and a follower, and **Multi-Objective Optimization**, where we derive the Pareto Front to visualize trade-offs between conflicting objectives. On the heuristic side, a **Genetic Algorithm** is implemented to solve a Jigsaw Puzzle, demonstrating the power of evolutionary computation in discrete search spaces.

Finally, we perform a comparative analysis of first-order and second-order methods, including an investigation into **Batch vs. Stochastic Gradient Descent**, the quadratic convergence of **Newton's Method**, and a practical implementation comparing **Backtracking Line Search** versus **RMSProp** on the challenging Rosenbrock function. This diverse set of problems highlights the theoretical foundations and practical nuances of optimization in modern learning systems.

Question 1: Line Search Optimization Problem

1.1 (a) General Form & Roles

The general line search iteration solves $\min_{\alpha>0} f(x_k + \alpha p_k)$ using the update rule:

$$x_{k+1} = x_k + \underbrace{\alpha_k}_{\substack{\text{Step Length} \\ \text{(How far)}}} \cdot \underbrace{p_k}_{\substack{\text{Search Direction} \\ \text{(Where to go)}}}$$

1.2 (b) Descent Direction

A vector p is a descent direction at x if moving along it reduces the objective function locally.

Necessary Condition: The angle between p and the gradient must be obtuse:

$$\boxed{\nabla f(x)^\top p < 0}$$

1.3 (c) Step Length Methods

- **Exact Line Search:** Solves for the global minimizer along the ray: $\alpha^* = \arg\min_{\alpha>0} \phi(\alpha)$.
- **Armijo / Backtracking:** Iteratively reduces a large initial α (e.g., $\alpha \leftarrow \tau\alpha$) until the *sufficient decrease condition* is met.
- **Wolfe Conditions:** Enforces both sufficient decrease (Armijo) and a *curvature condition* to ensure the step is not too short.
- **Fixed Step:** Uses a constant scalar (e.g., $\alpha_k = 10^{-3}$) for all iterations.

1.4 (d) Quadratic Function Analysis

Given: $f(x_1, x_2) = 2x_1^2 + x_1x_2 + 4x_2^2$, $\quad x^{(t)} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$, $\quad p^{(t)} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$.

1. Descent Direction Check

Compute gradient $\nabla f(x) = \begin{pmatrix} 4x_1 + x_2 \\ x_1 + 8x_2 \end{pmatrix}$. Evaluating at $x^{(t)}$:

$$\nabla f(x^{(t)}) = \begin{pmatrix} 0+1 \\ 0+8 \end{pmatrix} = \begin{pmatrix} 1 \\ 8 \end{pmatrix}$$

$$\text{Check Product:} \quad \nabla f(x^{(t)})^\top p^{(t)} = \begin{pmatrix} 1 & 8 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

$$= 1 - 16 = -15$$

$$-15 < 0 \implies \text{Descent Direction} \quad \checkmark$$

2. Formulate $\phi(\alpha)$

Define line function: $\phi(\alpha) = f(x^{(t)} + \alpha p^{(t)})$.

$$x(\alpha) = \begin{pmatrix} 0 \\ 1 \end{pmatrix} + \alpha \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} \alpha \\ 1 - 2\alpha \end{pmatrix}$$

Substitute into f:

$$\begin{aligned} \phi(\alpha) &= 2(\alpha)^2 + (\alpha)(1 - 2\alpha) + 4(1 - 2\alpha)^2 \\ &= 2\alpha^2 + \alpha - 2\alpha^2 + 4(1 - 4\alpha + 4\alpha^2) \\ &= \alpha + 4 - 16\alpha + 16\alpha^2 \end{aligned}$$

$$\boxed{\phi(\alpha) = 16\alpha^2 - 15\alpha + 4}$$

3. Exact Line Search (α^*)

Minimize the quadratic form by setting derivative to zero:

$$\phi'(\alpha) = \frac{d}{d\alpha}(16\alpha^2 - 15\alpha + 4) = 0$$

$$32\alpha - 15 = 0$$

$$\boxed{\alpha^* = \frac{15}{32}} \quad (> 0)$$

Question 2: Support Vector Machine (SVM)

2.1 (a) Hard-Margin Primal Optimization

The goal is to maximize the margin ($2/\|w\|$) while correctly classifying all training data. This is equivalent to minimizing the norm of w:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}||w||^2 \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1, \quad \forall i = 1, \dots, n \end{aligned}$$

2.2 (b) Lagrangian & Stationarity

We introduce Lagrange multipliers $\alpha_i \geq 0$ for the inequality constraints.

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2}||w||^2 - \sum_{i=1}^{n} \alpha_i [y_i(w^\top x_i + b) - 1]$$

To find the optimum, we take derivatives with respect to primal variables (w, b) and set them to zero:

- $\nabla_w \mathcal{L} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \Longrightarrow \boxed{w = \sum_{i=1}^{n} \alpha_i y_i x_i}$
- $\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0 \Longrightarrow \boxed{\sum_{i=1}^{n} \alpha_i y_i = 0}$

2.3 (c) Dual Problem

By substituting the expression for w back into the Lagrangian and using $\sum \alpha_i y_i = 0$, we obtain the Dual objective which depends only on α:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i^\top x_j) \\ \text{subject to} \quad & \alpha_i \geq 0, \quad \forall i \\ & \sum_{i=1}^{n} \alpha_i y_i = 0 \end{aligned}$$

2.4 (d) KKT Conditions & Support Vectors

Complementary Slackness: At optimality, the product of the multiplier and the constraint must be zero:

$$\alpha_i [y_i (w^\top x_i + b) - 1] = 0, \quad \forall i$$

Support Vectors: These are the data points x_i for which $\alpha_i > 0$. Due to slackness, they must satisfy $y_i(w^\top x_i + b) = 1$, meaning they lie exactly on the margin boundary.

Recovery:

- $w^* = \sum_{i \in SV} \alpha_i y_i x_i$
- b^* is recovered using any support vector k: $b^* = y_k - w^{*\top} x_k$.

2.5 (e) Soft Margin SVM

To handle non-separable data, we introduce slack variables $\xi_i \geq 0$ and a penalty parameter $C > 0$.

Primal Problem:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}||w||^2 + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \end{aligned}$$

Dual Constraints: The dual objective remains the same, but the constraints on α_i change to a "box constraint":

$$\boxed{0 \leq \alpha_i \leq C} \quad \text{and} \quad \sum_{i=1}^{n}\alpha_i y_i = 0$$

2.6 (f) Kernelization

The dual formulation depends only on dot products $x_i^\top x_j$. This allows us to map data to a higher-dimensional space $\phi(x)$ implicitly using a kernel function $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$.

The **kernelized decision function** becomes:

$$f(x) = \text{sign}\left(\sum_{i=1}^{n}\alpha_i y_i K(x_i, x) + b\right)$$

Question 3: Bilevel Optimization

3.1 Bilevel Formulation

The problem is modeled as a Stackelberg game where the leader (upper-level) minimizes F anticipating the follower's (lower-level) optimal response y.

$$\begin{aligned} \min_{x \geq 0} \quad & F(x, y) = (x-2)^2 + (y-3)^2 \\ \text{subject to} \quad & y \in \arg\min_{z \geq 0} \left\{ f(x, z) = (z - x^2)^2 + z \right\} \end{aligned}$$

3.2 Follower's Optimal Response

For a fixed strategy x chosen by the leader, the follower solves $\min_y f(x, y)$. Since f is convex with respect to y, we apply the first-order optimality condition ($\frac{\partial f}{\partial y} = 0$):

$$\begin{aligned} \frac{\partial f}{\partial y} = 2(y - x^2) + 1 &= 0 \\ 2y - 2x^2 &= -1 \\ \boxed{y^*(x) = x^2 - 0.5} & \end{aligned}$$

3.3 Reduction to Single-Level Problem

We substitute the follower's reaction function $y^*(x)$ into the leader's objective function $F(x, y)$ to eliminate y:

$$\begin{aligned}\tilde{F}(x) &= (x-2)^2 + (y^*(x) - 3)^2 \\ &= (x-2)^2 + ((x^2 - 0.5) - 3)^2 \\ \boxed{\tilde{F}(x) &= (x-2)^2 + (x^2 - 3.5)^2}\end{aligned}$$

3.4 Finding (x^*, y^*)

To find the leader's optimal x^*, we minimize $\tilde{F}(x)$ by setting its derivative to zero:

$$\begin{aligned}\frac{d\tilde{F}}{dx} = 2(x-2) + 2(x^2 - 3.5)(2x) &= 0 \\ (x-2) + 2x^3 - 7x &= 0 \\ 2x^3 - 6x - 2 &= 0 \\ \boxed{x^3 - 3x - 1 = 0}\end{aligned}$$

Solving this cubic equation for real roots (approximate solution):

$$x^* \approx 1.879$$

Now, we compute the corresponding y^* using the reaction function:

$$y^* = (1.879)^2 - 0.5 \approx 3.532 - 0.5 = \boxed{3.032}$$

Both x^* and y^* are non-negative, satisfying the constraints.

3.5 Regularity & Convexity

- **Convexity of Follower:** The second derivative of the follower's function is $\frac{\partial^2 f}{\partial y^2} = 2$. Since $2 > 0$, the function is strictly convex. This guarantees that for any x, the follower has a unique global minimizer $y^*(x)$.

- **Differentiability:** The reaction function $y^*(x) = x^2 - 0.5$ is continuous and differentiable (smooth). This regularity allows us to substitute it directly into the leader's problem and use gradient-based methods for the final solution.

Question 4: Multi-Objective Optimization (Pareto Front)

We consider a bi-objective optimization problem with:

$$f_1(x, y) = (x - 2)^2 + y^2$$
$$f_2(x, y) = x^2 + (y - 3)^2$$

4.1 (a) Pareto Optimality Definition

A feasible point (x^*, y^*) is said to be **Pareto Optimal** if there exists no other feasible point (x, y) such that:

1. $f_i(x, y) \leq f_i(x^*, y^*)$ for all $i = 1, 2$, and
2. $f_j(x, y) < f_j(x^*, y^*)$ for at least one objective j.

Intuitively, this means we cannot improve one objective without degrading the other.

4.2 (b) Weighted-Sum Scalarization

We form a scalar objective function using a weight $w \in [0, 1]$:

$$F_w(x, y) = wf_1(x, y) + (1 - w)f_2(x, y)$$

1. Derivation of Optimal Trajectory: By setting the gradients with respect to x and y to zero:

$$\frac{\partial F_w}{\partial x} = 2w(x-2) + 2(1-w)x = 0 \implies wx - 2w + x - wx = 0$$
$$\Rightarrow \boxed{x(w) = 2w}$$
$$\frac{\partial F_w}{\partial y} = 2wy + 2(1-w)(y-3) = 0 \implies wy + y - 3 - wy + 3w = 0$$
$$\Rightarrow \boxed{y(w) = 3(1-w)}$$

2. Geometry of the Pareto Set: As w varies from 0 to 1, the parametric solutions $(x(w), y(w))$ trace a straight line segment connecting the minimizer of f_2 at $(0, 3)$ to the minimizer of f_1 at $(2, 0)$.

4.3 (c) Pareto Front Expression

To find the relation between the objectives $u = f_1$ and $v = f_2$, we substitute the optimal path back into the functions:

$$u = (2w-2)^2 + (3(1-w))^2 = 4(1-w)^2 + 9(1-w)^2 = 13(1-w)^2$$
$$v = (2w)^2 + (3(1-w)-3)^2 = 4w^2 + 9w^2 = 13w^2$$

Taking the square root of both sides eliminates w:

$$\sqrt{u} = \sqrt{13}(1-w), \quad \sqrt{v} = \sqrt{13}w$$

Adding them yields the explicit equation of the Pareto Front:

$$\boxed{\sqrt{u} + \sqrt{v} = \sqrt{13}}$$

4.4 (d) Epsilon-Constraint Method

Alternatively, we can minimize f_1 subject to $f_2 \leq \epsilon$. The Lagrangian for this problem is:

$$L(x, y, \lambda) = f_1(x, y) + \lambda(f_2(x, y) - \epsilon) \tag{1}$$

The stationarity condition $\nabla f_1 + \lambda \nabla f_2 = 0$ is mathematically equivalent to the weighted sum condition $\nabla(w f_1 + (1 - w) f_2) = 0$ if we set $\lambda = \frac{1-w}{w}$. Thus, varying the constraint bound ϵ generates the same Pareto optimal points as varying the weight w.

4.5 (e) Numerical Values

The optimal decision variables and objective values for selected weights are:

Table 1: Optimal decision variables and objective values for selected weights

w	$x(w)$	$y(w)$	f_1	f_2
0.00	0.00	3.00	13.00	0.00
0.25	0.50	2.25	7.31	0.81
0.50	1.00	1.50	3.25	3.25
0.75	1.50	0.75	0.81	7.31
1.00	2.00	0.00	0.00	13.00

Question 5: Genetic Algorithm (Jigsaw Puzzle Solver)

This problem addresses the reconstruction of an image from a set of scrambled square pieces using an evolutionary approach. The puzzle consists of $N \times M$ pieces, and the objective is to find the optimal permutation that minimizes the visual discontinuity across boundaries.

5.1 (a) Pairwise Fit Score Definition

To numerically measure the compatibility between two pieces, we define a distance metric based on edge similarity.

- **Color Space Transformation:** Instead of the standard RGB space, we utilize the **CIELAB (Lab)** color space. In RGB, the Euclidean distance does not always correspond to perceptual color differences. The Lab space is perceptually uniform, meaning that geometric distances in this space map linearly to human visual perception, significantly improving edge matching accuracy.

- **Edge Dissimilarity Metric:** Let E_i^{right} be the vector of pixel values along the right edge of piece i, and E_j^{left} be the left edge of piece j. The dissimilarity cost $D(i, j)$ is calculated as the mean Euclidean distance:

$$D(i,j) = \frac{1}{H} \sum_{k=1}^{H} ||E_i^{right}(k) - E_j^{left}(k)||_2$$

  where H is the resolution of the edge (number of pixels). A lower D implies a better fit.

5.2 (b) Global Fitness Evaluation

The quality of a complete arrangement (individual) is evaluated by summing the dissimilarity costs of all adjacent boundaries in the grid. For an arrangement A represented as a grid, the Total Cost is:

$$\mathcal{C}(A) = \sum_{r,c} D_{LR}(A_{r,c}, A_{r,c+1}) + \sum_{r,c} D_{UD}(A_{r,c}, A_{r+1,c})$$

Since the Genetic Algorithm seeks to maximize an objective function, we define the **Fitness Score** as the inverse of the cost:

$$Fitness(A) = \frac{K}{1 + \mathcal{C}(A)}$$

where K is a scaling constant.

5.3 (c) Selection Mechanism

To select parents for the next generation, we employ **Tournament Selection**. This method is preferred over Roulette Wheel selection as it allows for adjustable selection pressure and handles negative or scaled fitness values robustly.

Process:

1. Randomly select a subset of k individuals from the population.
2. Identify the individual with the highest fitness in this subset.
3. Select this individual as a parent.

This ensures that better solutions have a higher probability of reproduction while maintaining genetic diversity.

5.4 (d) Evolutionary Engine & Local Search

We implement a hybrid evolutionary algorithm (Memetic Algorithm) to solve the puzzle effectively:

1. **Initialization:** A population of random permutations is generated.

2. **Crossover (Recombination):** We use **Order Crossover (OX1)**. Since the problem is a permutation problem (TSP-like), standard crossover would produce invalid solutions (duplicate or missing pieces). OX1 preserves the relative order of a subsequence from one parent and fills the remaining spots from the other parent.

3. **Mutation:** To prevent premature convergence to local optima, we apply **Swap Mutation**, where the positions of two random pieces are exchanged with a probability P_m.

4. **Elitism:** The best individuals from each generation are directly copied to the next to ensure monotonic improvement.

5. **Hybridization (Local Search):** Genetic algorithms are effective at global exploration but may struggle with fine-tuning the final solution (e.g., correcting two swapped sky pieces). To address this, we integrate a **Greedy Local Search** mechanism. At the end of the evolution (or periodically), the best candidate undergoes a refinement process where every pair of pieces is tentatively swapped. If the swap improves the fitness, the change is kept. This guarantees convergence to a locally optimal arrangement.

Question 7: Batch vs. Stochastic Gradient Descent

This section compares Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD) based on computational cost, gradient estimation, and convergence behavior.

7.1 (a) Computational Load per Iteration

- **Batch Gradient Descent (BGD):** In every iteration, BGD computes the gradient using the entire training dataset of size n. Consequently, the computational cost per step is $O(n)$. For very large datasets, this becomes computationally expensive and slow.
- **Stochastic Gradient Descent (SGD):** SGD updates parameters using the gradient computed from only a single training example chosen at random. The cost per step is $O(1)$, making each iteration extremely fast and independent of the dataset size.

7.2 (b) Noise (Variance) in Gradient Estimation

- **BGD:** Since it averages over all samples, the calculated gradient is the true gradient of the loss function. It has zero variance (deterministic given the data), resulting in a smooth trajectory towards the optimum.
- **SGD:** The gradient is approximated using a single sample, which introduces significant noise. The variance of the gradient estimate is high, causing the optimization path to be erratic and ”noisy” rather than a straight line.

7.3 (c) Stability and Convergence

- **BGD:** It provides stable convergence and moves directly ”downhill”. It is guaranteed to converge to a local minimum (or global minimum for convex functions) with a fixed learning rate.

- **SGD:** Due to the noisy gradient, the path oscillates. It does not converge to the exact minimum but rather wanders around it (fluctuates). To ensure convergence to the exact minimum, the learning rate must be decayed over time (e.g., $\alpha_k \to 0$).

Question 8: Steepest Descent Direction via First-Order Taylor

We aim to find the direction vector p (where $||p|| = 1$) that minimizes the function value in a local neighborhood of x_k. Using the first-order Taylor expansion for a small step size $\alpha > 0$:

$$f(x_k + \alpha p) \approx f(x_k) + \alpha \nabla f(x_k)^\top p$$

To maximize the decrease, we want to minimize the term $\nabla f(x_k)^\top p$. Using the geometric definition of the dot product:

$$\nabla f(x_k)^\top p = ||\nabla f(x_k)|| \cdot ||p|| \cdot \cos(\theta) = ||\nabla f(x_k)|| \cos(\theta)$$

where θ is the angle between the gradient and the direction p.
The quantity is minimized when $\cos(\theta) = -1$, which implies $\theta = 180^\circ$. This means p must be in the opposite direction of the gradient. Thus, the steepest descent direction is:

$$\boxed{p = -\frac{\nabla f(x_k)}{||\nabla f(x_k)||}}$$

Question 9: Pseudoinverse as Least-Squares Minimizer

We want to show that $x^* = (A^\top A)^{-1} A^\top b$ minimizes the objective function $L(x) = ||Ax - b||_2^2$.
First, let's expand the loss function:

$$\begin{aligned} L(x) &= (Ax - b)^\top (Ax - b) \\ &= (x^\top A^\top - b^\top)(Ax - b) \\ &= x^\top A^\top Ax - x^\top A^\top b - b^\top Ax + b^\top b \end{aligned}$$

Since $x^\top A^\top b$ is a scalar, it equals its transpose $(x^\top A^\top b)^\top = b^\top Ax$. Thus, the two middle terms are identical:

$$L(x) = x^\top A^\top Ax - 2x^\top A^\top b + b^\top b$$

To find the minimizer, we compute the gradient with respect to x and set it to zero:

$$\nabla_x L(x) = 2A^\top Ax - 2A^\top b$$

Setting $\nabla_x L(x) = 0$ (Normal Equations):

$$\begin{aligned} 2A^\top Ax - 2A^\top b &= 0 \\ A^\top Ax &= A^\top b \end{aligned}$$

Assuming $A^\top A$ is invertible (full column rank), we multiply by $(A^\top A)^{-1}$ from the left:

$$\boxed{x^* = (A^\top A)^{-1} A^\top b}$$

Question 10: Newton and Quasi-Newton Methods

Newton's method uses second-order information (curvature) to find stationary points faster than gradient descent, but it comes with computational challenges.

10.1 (a) Newton Step Computation

The Newton step is derived from the second-order Taylor approximation. The update rule is:

$$x_{k+1} = x_k - [H(x_k)]^{-1} \nabla f(x_k)$$

where $H(x_k) = \nabla^2 f(x_k)$ is the Hessian matrix and $\nabla f(x_k)$ is the gradient.

10.2 (b) Failure Cases

Newton's method seeks a point where the gradient is zero ($\nabla f(x) = 0$).

- **Attraction to Maxima/Saddles:** Since the method does not distinguish between types of stationary points, if initialized near a local maximum or a saddle point, it may converge to them instead of a minimum.
- **Divergence:** If the Hessian is singular (non-invertible) or if the starting point is too far from the optimum, the step size can become excessively large, causing divergence.

10.3 (c) Non-Descent Direction

The Newton direction $p_k = -H_k^{-1}\nabla f_k$ is a descent direction only if:

$$\nabla f_k^\top p_k < 0 \iff -\nabla f_k^\top H_k^{-1}\nabla f_k < 0$$

This condition holds if and only if the Hessian H_k is **Positive Definite** (conceptually, the function must be locally convex like a bowl). If H_k has negative eigenvalues (concave regions or saddle points), the Newton step may point uphill (ascent direction).

10.4 (d) Quasi-Newton Idea

The main idea is to replace the expensive Hessian matrix H (or its inverse H^{-1}) with an approximation matrix B_k.

- B_k is updated iteratively using only gradient information from previous steps.
- This avoids the $O(n^2)$ cost of computing the full Hessian and the $O(n^3)$ cost of inverting it, while maintaining a super-linear convergence rate (faster than GD).

10.5 (e) The Secant Equation

Quasi-Newton methods (like BFGS) update the approximate Hessian B_{k+1} such that it satisfies the **Secant Equation**:

$$B_{k+1}s_k = y_k$$

where:

- $s_k = x_{k+1} - x_k$ (change in position)

- $y_k = \nabla f_{k+1} - \nabla f_k$ (change in gradient)

This mimics the property of the true Hessian ($H\Delta x \approx \Delta(\nabla f)$) using finite differences.

Question 11: Quadratic Convergence of Newton's Method

11.1 (a) Quadratic vs. Linear Convergence

Let $e_k = ||x_k - x^*||$ be the error at iteration k.

- **Linear Convergence:** The error decreases by a constant factor in each step:

$$\lim_{k\to\infty} \frac{e_{k+1}}{e_k} = \mu \in (0, 1)$$

(e.g., Gradient Descent).

- **Quadratic Convergence:** The error at the next step is proportional to the **square** of the current error:

$$\lim_{k\to\infty} \frac{e_{k+1}}{e_k^2} = C > 0$$

This implies the number of correct digits roughly doubles with each iteration (e.g., Newton's Method).

11.2 (b) Newton Step Approximates Exact Displacement

We start with the optimality condition $\nabla f(x^*) = 0$. Using the first-order Taylor expansion of the gradient $\nabla f(x)$ around the current point x_k:

$$\nabla f(x^*) \approx \nabla f(x_k) + \nabla^2 f(x_k)(x^* - x_k)$$

Since $\nabla f(x^*) = 0$ and defining $H_k = \nabla^2 f(x_k)$:

$$0 \approx \nabla f(x_k) + H_k(x^* - x_k)$$

Rearranging terms to solve for the displacement $(x^* - x_k)$:

$$H_k(x^* - x_k) \approx -\nabla f(x_k)$$
$$x^* - x_k \approx -H_k^{-1}\nabla f(x_k)$$

The term $-H_k^{-1}\nabla f(x_k)$ is exactly the Newton step p_k.

Conclusion: For points close to the solution (where the quadratic approximation is good), the Newton step p_k is approximately equal to the exact displacement vector to the minimizer $(x^* - x_k)$.

Question 14: Step-Size Strategies in Gradient Descent

In this question, we analyze the impact of different step-size (learning rate) selection strategies on the performance of the Gradient Descent algorithm. The *Rosenbrock* function is used as a test case due to its narrow, curved valley, which poses a challenge for convergence.

14.1 (a) Description of Step-Size Methods

1. Backtracking Line Search (Armijo Rule): This method ensures that every step taken results in a "sufficient" decrease in the objective function value. In each iteration, a large initial step size (α) is chosen. If the following condition (Armijo condition) is not met, the step size is reduced by a factor $\rho \in (0, 1)$:

$$f(x_k - \alpha \nabla f_k) \leq f(x_k) - c \cdot \alpha ||\nabla f_k||^2$$

This strategy prevents severe oscillations and guarantees convergence, though the computational cost per iteration can be high due to multiple function evaluations.

2. Adaptive Learning Rate (RMSProp): Adaptive methods like RMSProp are designed to handle varying scales across different dimensions. This method adjusts the learning rate for each parameter individually:

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$$

$$x_{t+1} = x_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

By dividing the update by the moving average of the squared gradients, step sizes

are reduced in directions with steep slopes (to dampen oscillation) and increased in flat directions (to accelerate movement). This is particularly beneficial for the Rosenbrock function's flat valley.

14.2 (b) & (c) Implementation and Comparison

Both algorithms were executed on the function $f(x, y) = 100(y-x^2)^2+(1-x)^2$ starting from $x_0 = [-1.2, 1.0]$.

Analysis of Observations:

- **Trajectory:** The **Backtracking** method follows a cautious, zig-zag path towards the minimum. In contrast, **RMSProp** adapts quickly to the valley's curvature, taking a more direct trajectory toward the global optimum $(1, 1)$.
- **Convergence Speed:** The convergence plot (log scale) indicates that while Backtracking shows a rapid initial decrease, it slows down significantly at the valley floor where the gradient is small. RMSProp, due to its adaptive nature, maintains its speed in flat regions and demonstrates faster final convergence.

References

1. Najjar Aarabi, B., (Fall 2025). *Review of Optimization*. Lecture Slides, Machine Learning Course, University of Tehran.

2. Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer. (Reference for Line Search, Newton's Method, and Gradient Descent strategies).

3. Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press. (Reference for Lagrangian Duality, KKT conditions, and Least Squares).

4. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Reference for Support Vector Machines formulation).

5. Deb, K. (2001). *Multi-Objective Optimization using Evolutionary Algorithms*. John Wiley & Sons. (Reference for Pareto Optimality and Genetic Algorithms concepts).

6. Scikit-Image Documentation. Available at: `https://scikit-image.org/`. (Used for image processing in the Jigsaw Puzzle implementation).